

Healthcare System Simulator

Project Report

Course: Advanced Programming Concepts (APC)

Group Members:

Akshit Bahl | Divyansh Singh | Hiba Chehab

May 2026

1. Introduction

This report documents the design and implementation of a Healthcare System Simulator developed as a group project for the Advanced Programming Concepts course. The system simulates a network of hospitals, pharmacies, doctors, and patients, and operates in two modes: Normal Operations and Pandemic Response.

The project is structured as a client-server application with a Python simulation engine (backend) exposing a REST API via FastAPI, and a Next.js (React) dashboard (frontend) that visualizes the simulation state. The primary objective is to demonstrate Object-Oriented Programming principles (inheritance, encapsulation, polymorphism, composition) alongside functional programming constructs (map, filter, reduce, list comprehensions).

2. Project Structure

The project is organized into three logical layers:

```
PythonHealthcareSystemSimulationProject/
Main.py                # Launcher (starts both servers)
requirements.txt       # Python dependencies
backend/
  app.py               # FastAPI app + CORS config
  config.py            # Global constants
  api/routes.py        # REST API endpoints
  engine/
    simulation.py      # Core simulation engine
    pandemic.py        # Pandemic infection logic
  models/
    person.py          # Person, Patient, Doctor
    facility.py        # Facility, Hospital, Pharmacy
    simulation_mode.py  # SimulationMode (Strategy)
frontend/src/app/
  page.js              # Dashboard
  hospitals/patients/pharmacy/pandemic/ # Sub-pages
  components/          # Reusable UI components
  hooks/useSimulation.js # API communication hook
```

3. OOP Design and Class Hierarchy

Person Hierarchy (backend/models/person.py)

- Person - Base class with shared attributes (id, name, age, gender, contact) and a get_info() serialization method.
- Patient(Person) - Extends Person with health_status (Healthy/Infected/Deceased), immunity, days_infected, treatments_received, admitted status. Key methods: infect(), update_health(), admit(), discharge(), treat().
- Doctor(Person) - Extends Person with assigned_facility, assigned_patients, max_patients. Methods: assign_patient(), remove_patient(), diagnose().

Facility Hierarchy (backend/models/facility.py)

- Facility - Base class with id, name, location, facility_type, active. Provides get_status().
- Hospital(Facility) - Bed management with total_beds, base_beds. Uses @property for computed attributes (occupied_beds, available_beds). Methods: admit_patient(), discharge_patient(), apply_surge_capacity().
- Pharmacy(Facility) - Medication inventory (dict mapping names to stock counts). Methods: fill_prescription(), reset_daily_count().

Other Classes

- **SimulationMode** (backend/models/simulation_mode.py) - Implements a Strategy-like pattern with mode-specific parameters (infection_rate, surge_capacity_multiplier). Methods: set_mode(), toggle(), is_pandemic property.
- **HealthcareSimulation** (backend/engine/simulation.py) - Central orchestrator composing all entities. Manages the full simulation lifecycle via daily tick() calls.
- **PandemicEngine** (backend/engine/pandemic.py) - Handles pandemic-specific logic: infection seeding, daily spread, SIR counting, death tracking.

OOP Principles Demonstrated

Principle	Where Applied
Inheritance	Patient/Doctor extend Person; Hospital/Pharmacy extend Facility
Encapsulation	Each class manages its own state (e.g., Patient.treat() handles immunity/status)
Polymorphism	Patient and Doctor override to_dict() from Person.get_info()
Composition	HealthcareSimulation composes Patient, Doctor, Hospital, Pharmacy lists
Properties	Hospital.occupied_beds/@property for computed read-only attributes

4. Functional Programming Concepts

- **map()** - Used in get_state() to serialize all entities: list(map(lambda p: p.to_dict(), self.patients)). Also in tick() to apply health updates to all patients.
- **filter()** - Used in tick() to find infected unadmitted patients for admission and healthy/deceased admitted patients for discharge. Also in PandemicEngine for susceptible patients.
- **reduce()** - Used in get_state() to aggregate total occupied beds, capacity, and medication stock across all hospitals and pharmacies via functools.reduce.
- **List Comprehensions** - Used extensively: filtering available hospitals, finding doctors at a facility, counting statuses, building inventory lists, constructing API responses.

5. Core Functionalities

5.1 Simulation Engine

The simulation engine initializes a population of 50 patients, 12 doctors, 3 hospitals, and 2 pharmacies. Each tick() call advances the simulation by one day:

1. Reset daily pharmacy prescription counters.
2. Pandemic seeding: On the first pandemic tick, 5 healthy patients are infected and hospital beds increase by 50%.
3. Health updates: In normal mode, 2% of healthy patients are infected. In pandemic mode, 15% infection rate via PandemicEngine.
4. Admissions: Infected non-admitted patients are admitted to the first hospital with available beds and doctors.
5. Treatment: Each admitted infected patient receives 1 medication and immunity +0.10. Recovery if immunity > 0.50, death after 2 failed treatments.
6. Discharges: Healthy or deceased patients are discharged.

5.2 Normal vs. Pandemic Mode

Parameter	Normal	Pandemic
Infection rate	2%	15%
Surge capacity	1.0x	1.5x

5.3 REST API Endpoints

Method	Endpoint	Description
GET	/api/status	Full simulation state snapshot
GET	/api/hospitals	All hospitals with occupancy data
GET	/api/hospitals/{id}	Single hospital with patients/doctors
GET	/api/patients	All patients with health status
GET	/api/doctors	All doctors with assignments
GET	/api/pharmacy	Pharmacy inventory status
POST	/api/mode	Switch Normal/Pandemic mode
POST	/api/simulation/tick	Advance simulation by one day
POST	/api/simulation/reset	Reset simulation to Day 0

5.4 Frontend Dashboard

The frontend is a multi-page Next.js application with five views: Dashboard (KPI cards and bed occupancy chart), Hospitals (per-hospital detail cards with occupancy bars), Patients (sortable/filterable registry table), Pharmacy (medication inventory grid), and Pandemic (SIR compartment cards and infection statistics). A custom React hook (useSimulation) handles all API communication.

5.5 Launcher Script (Main.py)

Main.py provides a single-command launch: it checks/installs Python and Node dependencies, starts the FastAPI backend on port 8000, starts Next.js on port 3000, opens the browser, and gracefully terminates both servers on exit.

6. Group Member Contributions

The work was distributed among three group members. The focus below is on backend contributions, as OOP is the core concept of this course.

Akshit Bahl

- OOP Model Design: Designed the class hierarchy - Person base class and its subclasses Patient and Doctor (backend/models/person.py), including health state machine logic (update_health, treat, infect), immunity system, and admission/discharge workflow.
- Facility Classes: Implemented Facility base class and subclasses Hospital and Pharmacy (backend/models/facility.py), including bed management with @property decorators, occupancy rate calculation, and pharmacy inventory system.
- UML Class Diagram: Created the UML class diagram documenting the OOP hierarchy.
- Project Setup: Set up the project structure, Git repository, and Main.py launcher.

Divyansh Singh

- Simulation Engine: Implemented HealthcareSimulation (backend/engine/simulation.py) - daily tick() loop, patient population generation, doctor assignment, hospital admission logic, treatment workflow, discharge management.
- Functional Programming: Implemented all map(), filter(), reduce(), and list comprehension usages across the simulation engine and state serialization.
- REST API: Designed FastAPI routes (backend/api/routes.py), app setup with CORS (backend/app.py), and configuration module (backend/config.py).
- Frontend: Built all five dashboard pages, reusable components (StatCard, BedChart, PatientTable, Sidebar), useSimulation hook, and CSS design system.

Hiba Chehab

- SimulationMode (Strategy Pattern): Designed SimulationMode class (backend/models/simulation_mode.py) with mode parameter dictionary, set_mode()/toggle() methods, and is_pandemic property.
- Pandemic Engine: Implemented PandemicEngine class (backend/engine/pandemic.py) - infection seeding, daily spread logic, SIR compartment counting, and death tracking.
- Mode Switching Logic: Implemented mode-switching workflow in HealthcareSimulation.set_mode(), including pandemic state reset and surge capacity activation/deactivation.
- Testing and Documentation: Tested the simulation across different scenarios, verified edge cases, and contributed to project documentation.

7. Required Packages and Materials

Python Backend Dependencies (requirements.txt)

Package	Version	Purpose
fastapi	latest	High-performance async web framework for REST API
uvicorn	latest	ASGI server to run the FastAPI application
pydantic	latest	Data validation and serialization (FastAPI dep)

Python Standard Library modules used: uuid, random, functools (reduce), typing, os, sys, time, subprocess, webbrowser, pathlib.

Frontend Dependencies (package.json)

Package	Version	Purpose
next	16.2.3	React-based framework for the web dashboard
react	19.2.4	UI component library
react-dom	19.2.4	React DOM rendering
recharts	^3.8.1	Chart library for bed occupancy visualization
eslint	^9	Code linting (dev dependency)

System Requirements

- Python 3.10 or later
- Node.js (LTS recommended) and npm
- A modern web browser

Supplementary Materials

File	Description
Main.py	Unified launcher script - starts both backend and frontend
UML_class.png	UML class diagram of the OOP hierarchy
README.md	Project overview with quick-start instructions
project_workflow.md	Detailed workflow documentation

8. How to Run

```
# Navigate to the project directory
cd PythonHealthcareSystemSimulationProject

# Run everything with a single command:
python Main.py
```

The launcher will automatically install any missing dependencies and open the dashboard at <http://localhost:3000>. The FastAPI backend runs at <http://localhost:8000> with interactive API docs at <http://localhost:8000/docs>.

9. Conclusion

This project demonstrates a comprehensive application of Object-Oriented Programming principles in Python through a healthcare simulation system. The class hierarchy models real-world entities (Person, Patient, Doctor, Facility, Hospital, Pharmacy) with proper inheritance, encapsulation, and composition. Functional programming constructs (map, filter, reduce, list comprehensions) are used throughout the simulation engine. The Strategy pattern, implemented via `SimulationMode`, allows dynamic switching between Normal and Pandemic modes at runtime. The REST API and web dashboard provide a clear interface for interacting with and visualizing the simulation state.